

ADVANCED MACHINE LEARNING · WORKSHEET 06

Stochastic & Mini-Batch Gradient Descent

Lecture 6

Name	Branch / Year	Date
_____	_____	_____

HOW TO USE THIS SHEET

Six recurring box styles: **HOOK** (▷) opens a motivating question; **KEY INSIGHT** (★) flags a core idea worth remembering; **WARNING** (△) flags a common mistake; **PRACTICE** is a problem you solve by hand; **THINK & DISCUSS** (?) is a short pause-and-answer prompt; **TAKEAWAY** (◆) closes out a section.

Part	Sections	What you will be doing
I	1	See exactly why Batch GD updates so rarely.
II	2–3	Build SGD from a single random sample and compare it to BGD term by term.
III	4–6	See it (plots), run it by hand (dry run), and code it (pseudocode).
IV	7–9	Weigh SGD's strengths against its noise, then ask what a compromise looks like.
V	10–12	Meet Mini-Batch GD, run it by hand, and compare all three methods side by side.

NOTATION USED THROUGHOUT THIS SHEET

m	number of training examples
d	number of features (parameter dimension)
θ	parameter vector, e.g. (θ_0, θ_1) for a line
α	learning rate
$J_i(\theta)$	loss from a single training example i
$J(\theta)$	average training cost over the full dataset
Iteration	one parameter update
Epoch	one complete pass over all m training examples

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^m \quad J(\theta) = \frac{1}{m} \sum_{i=1}^m J_i(\theta)$$

SECTION 1 Revision and Motivation

▷ HOOK

In Batch Gradient Descent (BGD), we compute the gradient of the average cost over all m examples and then update θ **once**. The rhythm is: scan all samples $\rightarrow$ average their gradients $\rightarrow$ update once. Must we always wait for the entire dataset before one update?

If $m = 10,000,000$, how many parameter updates happen in one BGD epoch? **One**. Is one update worth scanning 10 million samples every time?

Why BGD is attractive:

- **Stable direction:** Every sample contributes, so noisy gradients average out.

- **Deterministic path:** Steps are exactly repeatable for the same initialization.
- **Smooth convergence:** Cost usually decreases smoothly on simple convex losses.
- **Good for small datasets:** Exact gradients are affordable when data fits in memory.

Disadvantages as real problems:

Limitation	Why it happens	When it's serious	Example
Waits before update	Average over all m	Full pass takes hours	Ad clicks model (100M rows)
High cost/update	Needs grads for every sample	Large m and d	Image data (millions of pixels)
Memory/IO pressure	Data read repeatedly	Data too large for RAM	Recommendation system logs
Slow feedback	Learns only after full scan	Data changes quickly	Real-time fraud detection
Redundant work	Similar samples = similar grads	Repeated patterns	Identical ad impressions

All these problems have one root cause: **BGD demands an exact full-data gradient before it moves.** Exactness is useful, but for huge data it can be too expensive.

PRACTICE P1 · SCENARIO MATCHING

Draw lines to match the specific dataset scenario with the primary limitation it causes for Batch Gradient Descent.

Scenario

Primary BGD Limitation

- | | |
|---|----------------------------|
| 1. A recommendation model trained on 500GB of user-item logs. | A. Slow feedback |
| 2. A fraud detection system that needs to adapt immediately to new incoming transactions. | B. Memory and I/O pressure |
| 3. A dataset with 10 million identical ad impressions. | C. Redundant work |

Answers: 1 → B 2 → A 3 → C

This raises the natural next question: can we update θ without waiting for the entire dataset? That single question is the seed of everything in the next section.

SECTION 2 Stochastic Gradient Descent (SGD)

? THINK & DISCUSS

Think–Pair–Share: The full gradient is expensive to compute. What is the *smallest* amount of data that could still give you a direction to move θ in? One Sample

That choice immediately creates a new danger for the update. What is it?

Noisy updates

SGD is the answer to the waiting problem. Instead of asking the whole dataset for one perfect direction, SGD asks **one randomly chosen example** for a quick direction and updates immediately.

Analogy: Batch GD is like polling the *entire class* before making any decision. SGD is like asking *one random student*, acting on their answer immediately, then asking another. Any single answer might

be off, but decisions happen instantly instead of after a full roll call.

Core Intuition: Each training example gives a small opinion about how θ should move. A single opinion may be biased, but if selected randomly, many opinions average out toward the full-data direction. SGD does not change the loss function; it changes how we estimate the gradient before each update.

Let $J_i(\theta)$ be the loss from one training example. At iteration t , choose one sample index i_t **uniformly at random** from $\{1, \dots, m\}$ — every sample is equally likely to be picked. The stochastic update uses only that sample's gradient:

$$i_t \sim \text{Uniform}\{1, 2, \dots, m\}, \quad g_t = \nabla J_{i_t}(\theta_t), \quad \theta_{t+1} = \theta_t - \alpha g_t$$

In the middle expression, g_t is the direction used for update t :

- J_{i_t} is the loss for the training example selected at iteration t ;
- ∇J_{i_t} means the gradient (the vector of loss slopes) for that example;
- θ_t is the current parameter vector when the gradient is evaluated; and
- therefore $g_t = \nabla J_{i_t}(\theta_t)$ is the selected example's gradient, computed at the current parameters during iteration t .

★ KEY INSIGHT

SGD is **not** training on only one sample forever! One update uses one sample, but one *epoch* uses every sample once. If $m = 1000$, one SGD epoch contains 1000 updates.

After E full epochs, SGD has therefore made $E \times m$ updates in total — compare that to Batch GD's mere E updates over the same E epochs.

Complete SGD Workflow:

1. Initialize θ . Choose learning rate α and number of epochs.
2. At the start of each epoch, **shuffle** the training examples.
3. Visit samples one by one in the shuffled order.
4. For each sample, compute prediction, sample loss, sample gradient, and **immediately update** θ .
5. Repeat for multiple epochs, tracking the training (or validation) loss trend as you go.

△ WARNING SHUFFLING

Real datasets are often ordered. Without shuffling, consecutive SGD updates may follow the ordering pattern instead of the true learning signal. Randomization protects SGD from biased updates!

Effect of Learning Rate & Noisy Gradients:

- **Small α :** Safe but slow.
- **Large α :** Dangerous; a noisy gradient can cause a huge jump.
- **Constant α :** Simple to implement, but θ may keep bouncing near the optimum instead of settling into it.
- **Decreasing α :** Explore with larger steps early, settle with smaller steps later. (*Learning-rate schedules are studied in more depth in your Deep Learning course.*)

Note: A single SGD step may temporarily increase the full training loss because it follows one sample. The useful behavior is the downward trend over many random updates.

Misconceptions Corrected:

Misconception	Correction
SGD ignores most of the dataset.	One update uses one sample, but an epoch uses all.
Loss must decrease every update.	Full loss can temporarily increase after a noisy update.
More updates always means faster convergence.	Each update is cheaper but less accurate; wall-clock time and stability matter too.
Stochastic means random parameters.	Randomness is in sample selection; the rule is precise.

PRACTICE P2 · SGD WORKFLOW BLANKS

Fill in the blanks to complete the core concepts of SGD.

1. One SGD update uses one sample(s).
2. If your dataset has m examples, one SGD **epoch** contains m parameter updates.
3. We must shuffle the dataset at the start of each epoch to prevent biased update sequences.
4. An outlier sample can create a large gradient, leading to noisy updates.

SECTION 3 Mathematical Comparison: Batch GD vs SGD

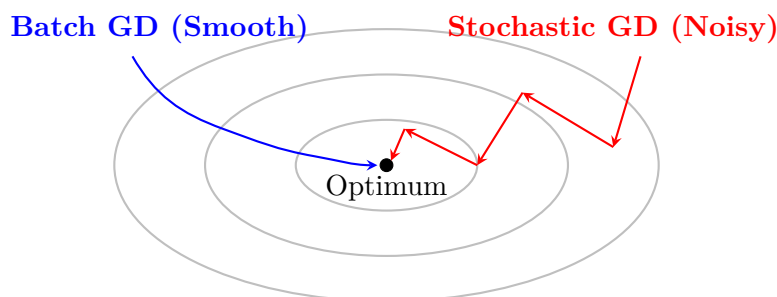
The key idea is simple: same objective, different gradient estimator.

Aspect	Batch GD	Stochastic GD
Gradient Used	Exact average over all m samples	Gradient of one randomly selected sample
Update Timing	After the full dataset has been processed	Immediately after one sample
Updates per Epoch	1	m
Cost per Update	$O(md)$	$O(d)$
Cost per Epoch	$O(md)$	$O(md)$ overall, delivered via m smaller updates
Memory Pattern	Needs access to full pass before update	Can stream one sample at a time
Path	Smooth and stable for well-behaved losses	Noisy but often fast in early progress

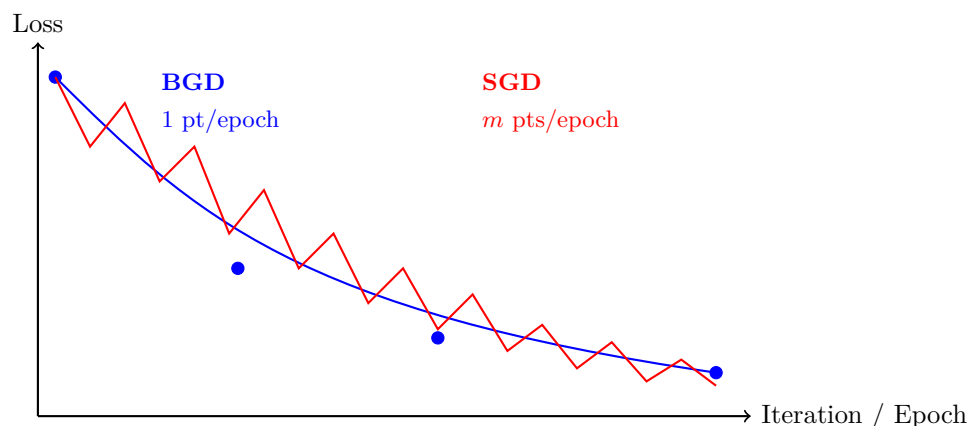
Why SGD Gradients are Noisy: The gradient contribution from sample i is the sample gradient. The full gradient is the average of all these contributions. SGD randomly selects one contribution, so the selected gradient is a **random variable**. It is unbiased (its average direction over many random choices equals the full-gradient direction), but one selected sample can still be far from that average. Outliers, class imbalance, and rare patterns increase this variance, leading to a zig-zag path.

Why Noise can sometimes help: In non-convex models like neural networks, optimization may get stuck in shallow local minima. The randomness in SGD can help the optimizer bounce out of these flat regions. However, too much noise can slow or even prevent convergence — the learning rate and batch size have to be chosen carefully. *You will study this trade-off in much greater depth in your Deep Learning course.*

SECTION 4 Visual Intuition



- **BGD convergence path:** Smooth arrows toward the center. Full gradients give stable, averaged directions.
 - **SGD zig-zag path:** Arrows jump left and right while trending inward. Single samples produce noisy directions.
 - **Loss vs epoch:** BGD has a smooth curve; SGD has a bumpy downward trend.
 - **Parameter-plane view:** The figure above is literally the θ_0 - θ_1 plane — every arrow is one real step of θ , and both paths are headed for the same optimum.
- Cost also looks different depending on what sits on the x-axis:



BGD needs only one x-axis step per epoch, and each step is smooth. **SGD** needs many x-axis steps — one per iteration — and each step is noisy, yet both curves settle into the same low-loss region. This is why SGD should be judged by its *trend*, not by any single point.

? THINK & DISCUSS

- Why don't Batch GD's arrows in the parameter-plane figure change direction wildly?
It averages gradients across all samples, cancelling out individual noise into stable direction.
- Can one "bad-looking" SGD step still be useful for training overall? Why?
Yes because noise helps in escaping local minima.
- When comparing how fast two training runs are, would epochs, iterations, or wall-clock time be the fairest x-axis? *Wall Clock time, because the actual computational cost per iteration and epoch drastically differs across methods.*

SECTION 5 Dry Run Example: One Epoch of BGD and SGD

Let's use a tiny linear regression dataset to see when θ actually changes. Initial values: $\theta_0 = 0, \theta_1 = 0, \alpha = 0.1$.

x_i	y_i
1	2
2	4
3	6
4	8

Batch GD (One Epoch): All predictions are calculated using the *initial* $\theta = [0, 0]^\top$.
For one example, use the half-squared loss

$$J_i(\theta) = \frac{1}{2}(\hat{y}_i - y_i)^2, \quad \hat{y}_i = \theta_0 + \theta_1 x_i.$$

If $e_i = \hat{y}_i - y_i$, the two gradient entries are

$$\frac{\partial J_i}{\partial \theta_0} = e_i, \quad \frac{\partial J_i}{\partial \theta_1} = e_i x_i.$$

WORKED FIRST ROW OF THE BATCH TABLE

For $i = 1$, $x_1 = 1$, $y_1 = 2$, and $\theta = [0, 0]^\top$:

$$\begin{aligned} \hat{y}_1 &= 0 + 0(1) = 0, & e_1 &= 0 - 2 = -2, \\ \text{Grad}(\theta_0) &= e_1 = -2, & \text{Grad}(\theta_1) &= e_1 x_1 = (-2)(1) = -2. \end{aligned}$$

Use these same two gradient rules to verify the remaining rows.

i	x_i	y_i	Prediction $\hat{y}_i$	Error $(\hat{y}_i - y_i)$	Grad θ_0	Grad θ_1
1	1	2	0	-2	-2	-2
2	2	4	0	-4	-4	-8
3	3	6	0	-6	-6	-18
4	4	8	0	-8	-8	-32
Average Gradient (Full Batch)					-5	-15

$$\text{Update: } \theta = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} -5 \\ -15 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 1.5 \end{bmatrix}. \quad (1 \text{ Update Total}).$$

? THINK & DISCUSS

Quick Check: Did θ change after Sample 1 was processed in the table above? [No] Justify:

Batch GD accumulates gradients across all training samples and updates θ only once at the end of epoch.

Stochastic GD (One Epoch): We shuffle the data to visit in order: Sample 2, Sample 4, Sample 1, Sample 3. Notice how θ changes *after every row*. The single-sample formula comes from differentiating the same half-squared loss used above. With $e_i = \hat{y}_i - y_i$ and $\hat{y}_i = \theta_0 + \theta_1 x_i$, the chain rule gives

$$\nabla J_i(\theta) = \begin{bmatrix} \partial J_i / \partial \theta_0 \\ \partial J_i / \partial \theta_1 \end{bmatrix} = \begin{bmatrix} e_i \\ e_i x_i \end{bmatrix} = \begin{bmatrix} (\hat{y}_i - y_i) \\ (\hat{y}_i - y_i) x_i \end{bmatrix}.$$

WORKED FIRST SGD UPDATE

For the first shuffled sample, $x_i = 2$, $y_i = 4$, and $\theta = [0, 0]^T$:

$$\hat{y}_i = 0 + 0(2) = 0, \quad e_i = \hat{y}_i - y_i = 0 - 4 = -4,$$

$$\nabla J_i(\theta) = \begin{bmatrix} -4 \\ (-4)(2) \end{bmatrix} = \begin{bmatrix} -4 \\ -8 \end{bmatrix}; \quad \theta_{\text{new}} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} -4 \\ -8 \end{bmatrix} = \begin{bmatrix} 0.4 \\ 0.8 \end{bmatrix}.$$

The next sample must use these new values, $\theta_0 = 0.4$ and $\theta_1 = 0.8$.

Step	Sample	x_i	y_i	Prediction	Sample Gradient	New θ
1	2	2	4	0.000	$\begin{bmatrix} -4.000 \\ -8.000 \end{bmatrix}$	$\begin{bmatrix} 0.400 \\ 0.800 \end{bmatrix}$
2	4	4	8	3.600	$\begin{bmatrix} -4.400 \\ -17.600 \end{bmatrix}$	$\begin{bmatrix} 0.840 \\ 2.560 \end{bmatrix}$
3	1	1	2	3.400	$\begin{bmatrix} 1.400 \\ 1.400 \end{bmatrix}$	$\begin{bmatrix} 0.700 \\ 2.420 \end{bmatrix}$
4	3	3	6	<u>7.96</u>	$\begin{bmatrix} 1.960 \\ 5.880 \end{bmatrix}$	$\begin{bmatrix} 0.504 \\ 1.832 \end{bmatrix}$

PRACTICE P3 · COMPLETE THE SGD DRY RUN

Using your calculator, fill in the missing cells for Step 4 in the SGD table above. *Hint: For Step 4, use the θ from Step 3 to calculate the new prediction!*

? THINK & DISCUSS

Look back at the SGD table: Sample 1's error (Step 3) came out **positive** (+1.400), while Samples 2 and 4 (Steps 1–2) both gave **negative** errors — yet all four points lie exactly on the line $y = 2x$. Why did the sign flip? _____

SECTION 6 Pseudocode

Compare the two algorithms side by side. Here is Batch GD, written as pseudocode:

REFERENCE · BATCH GRADIENT DESCENT

Input: dataset $D = \{(x_i, y_i)\}_{i=1}^m$, learning rate α , epochs E . **Initialize θ .**

```

for epoch = 1 to E:                                     outer epoch loop
    grad = 0
    for i = 1 to m:                                       sample loop for accumulation
        grad = grad +  $\nabla J_i(\theta)$ 
    grad = grad/m                                         full-batch average gradient
     $\theta \leftarrow \theta - \alpha \cdot \text{grad}$            one update per EPOCH
return  $\theta$ 

```

Now try to reconstruct **SGD**'s pseudocode yourself — the structure is almost identical, but one line moves.

PRACTICE P4 · REARRANGE THE PSEUDOCODE

The pseudocode lines for SGD are scrambled. Write them in the correct nested order by placing the letters in the blanks. Pay attention to the indentation!

- A. $\theta := \theta - \alpha \nabla J_i(\theta)$
- B. for epoch = 1 to E:
- C. $\nabla J_i(\theta) = \text{gradient_of_sample}(x_i, y_i, \theta)$
- D. shuffle(Dataset)
- E. for i = 1 to m:

BDECA**★ KEY INSIGHT**

Implementation warning: In SGD, do not accumulate gradients through the full sample loop before updating. That silently turns the code back into Batch GD.

? THINK & DISCUSS

Spot the Difference: Compare the Batch GD pseudocode above with the SGD pseudocode you just assembled. Exactly **one** line changes position between the two. Which line is it, and where does it move to? _____

SECTION 7 Advantages of SGD

- **Faster first update:** The model changes after one sample instead of after m samples. Learning begins immediately.
- **Lower memory usage:** Only one sample, the current θ , and one gradient are needed for an update.
- **Better scalability:** The per-update cost does not grow with m . This is useful when m is huge.
- **Suitable for massive datasets:** An approximate direction is often enough to start improving the model; waiting for the exact full gradient may be wasteful.
- **Better exploration:** Noisy gradients can help avoid getting stuck in flat or shallow regions in non-convex problems.

SECTION 8 Limitations of SGD

- **Oscillating convergence:** Different samples can push θ in conflicting directions.
- **Noisy updates:** An outlier sample can create a large gradient jump.
- **Sensitive learning rate:** α controls both step size and how strongly noise affects θ .
- **Many updates needed:** Each update is cheap but less informative.
- **Difficult stopping:** Iteration-level loss fluctuates rapidly.
- **Variance across runs:** Different shuffles can produce different paths.

SECTION 9 Motivation for Mini-Batch Gradient Descent

▷ HOOK

SGD solved the waiting problem but created noisy movement. If **one** sample is too noisy, and **all** samples together is too expensive — what is the middle ground? Can we cut the noise *without* going back to scanning the whole dataset every time?

Natural answer: average a **small group** of samples. This is Mini-Batch Gradient Descent. **Analogy:** instead of asking the whole class or one student, ask a small group. The group answer is less noisy than one person, but much faster than asking everyone.

SECTION 10 Mini-Batch Gradient Descent

Mini-batch GD splits the shuffled dataset into small groups called **mini-batches**. Each update uses the average gradient over one mini-batch of size b .

The batch size is the compromise dial: $b = 1$ behaves like SGD, $b = m$ behaves like Batch GD, and values in between trade some speed for a steadier direction. One epoch produces $\lceil m/b \rceil$ updates.

Workflow:

1. Shuffle the dataset at the start of each epoch.
2. Split it into mini-batches of size b .
3. Compute the average gradient over the first mini-batch and update θ immediately.
4. Continue until every sample has been used once.
5. Repeat for multiple epochs.

Practical points:

- **Typical batch sizes:** 16, 32, 64, 128, 256 — common choices, not compulsory ones.
- **Why powers of two:** GPU and hardware kernels often handle these sizes most efficiently; the math itself does not require them.
- **GPU utilization:** a single sample under-uses a GPU's parallel cores; a mini-batch gives it enough parallel work to stay busy.
- **Memory efficiency:** only the current mini-batch needs to sit in memory for an update — not the whole dataset.

REFERENCE · MINI-BATCH GRADIENT DESCENT

```

Input: dataset  $D = \{(x_i, y_i)\}_{i=1}^m$ , learning rate  $\alpha$ , epochs  $E$ , batch size  $b$ .   Initialize  $\theta$ .
for epoch = 1 to E:                                                                outer epoch loop
    shuffle( $D$ )
    split  $D$  into mini-batches  $B_1, B_2, \dots, B_K$ 
    for each mini-batch  $B_k$ :                                                        mini-batch loop
        grad = 0
        for each  $(x_i, y_i) \in B_k$ :                                                sample loop inside batch
            grad = grad +  $\nabla J_i(\theta)$ 
        grad = grad /  $|B_k|$ 
         $\theta \leftarrow \theta - \alpha \cdot \text{grad}$                                          one update per MINI-BATCH
return  $\theta$ 

```

SECTION 11 Dry Run of Mini-Batch GD

PRACTICE P5 · MINI-BATCH DRY RUN ($b = 2$)

Using the same 4-sample dataset from Section 5, let's run Mini-Batch GD with a batch size of $b = 2$. We shuffle to order: Sample 2, Sample 4, Sample 1, Sample 3.

Worked Iteration 1: Mini-Batch 1 = {Sample 2, Sample 4}

Start with $\theta = [0, 0]^\top$. Both predictions are 0, so

$$\mathbf{g}_2 = \begin{bmatrix} 0 - 4 \\ (0 - 4)(2) \end{bmatrix} = \begin{bmatrix} -4 \\ -8 \end{bmatrix}, \quad \mathbf{g}_4 = \begin{bmatrix} 0 - 8 \\ (0 - 8)(4) \end{bmatrix} = \begin{bmatrix} -8 \\ -32 \end{bmatrix}.$$

Average the two gradients, then update once:

$$\bar{\mathbf{g}}_1 = \frac{\mathbf{g}_2 + \mathbf{g}_4}{2} = \frac{1}{2} \begin{bmatrix} -12 \\ -40 \end{bmatrix} = \begin{bmatrix} -6 \\ -20 \end{bmatrix},$$

$$\theta^{(1)} = \theta^{(0)} - \alpha \bar{\mathbf{g}}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} -6 \\ -20 \end{bmatrix} = \begin{bmatrix} 0.6 \\ 2.0 \end{bmatrix}.$$

Your Turn: Mini-Batch 2 = {Sample 1, Sample 3} Using the new $\theta = [0.6, 2.0]^\top$, calculate the next update:

- Pred 1: $\hat{y}_1 = 0.6 + 2.0(1) = 2.6$. Error = $2.6 - 2 = 0.6$. Grad 1: $[0.6, 0.6]^\top$.
- Pred 3: $\hat{y}_3 = 0.6 + 2.0(3) = 6.6$. Error = $6.6 - 6 = 0.6$. Grad 3: $\begin{bmatrix} 0.6 \\ 1.8 \end{bmatrix}$.

Average Gradient for Batch 2: $\begin{bmatrix} 0.6 \\ 1.2 \end{bmatrix}$

Final θ after Epoch 1: $\begin{bmatrix} 0.54 \\ 1.88 \end{bmatrix}$

The update is less noisy than SGD because two gradients are averaged, but it still happens before the full dataset is processed.

HOMEWORK P6 · MINI-BATCH WITH AN UNEVEN SPLIT ($b = 3$)

Using the shuffled order (Sample 2, Sample 4, Sample 1, Sample 3), run Mini-Batch GD with $b = 3$. Because 4 is not divisible by 3, use

$$B_1 = \{S_2, S_4, S_1\}, \quad B_2 = \{S_3\}.$$

Update 1 (provided): Start from $\theta^{(0)} = [0, 0]^\top$.

$$\bar{\mathbf{g}}_1 = \frac{1}{3} \left(\begin{bmatrix} -4 \\ -8 \end{bmatrix} + \begin{bmatrix} -8 \\ -32 \end{bmatrix} + \begin{bmatrix} -2 \\ -2 \end{bmatrix} \right) = \begin{bmatrix} -4.667 \\ -14.000 \end{bmatrix},$$

$$\theta^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} -4.667 \\ -14.000 \end{bmatrix} = \begin{bmatrix} 0.467 \\ 1.400 \end{bmatrix}.$$

Update 2 (complete for homework): B_2 contains only Sample 3.

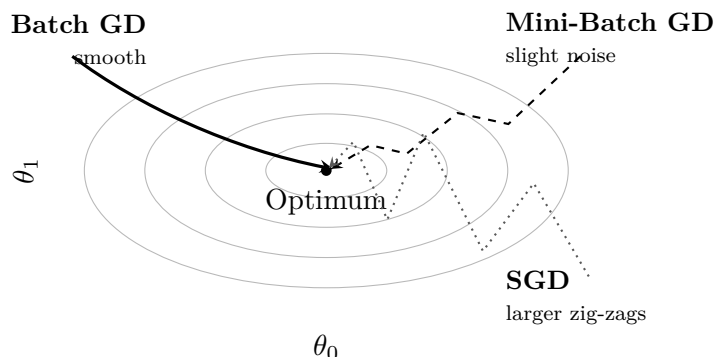
$$\hat{y}_3 = 0.467 + 1.400(3) = 4.667, \quad e_3 = \hat{y}_3 - 6 = -1.333,$$

$$\bar{\mathbf{g}}_2 = \begin{bmatrix} -1.333 \\ -4.000 \end{bmatrix}, \quad \theta^{(2)} = \begin{bmatrix} 0.467 \\ 1.800 \end{bmatrix}.$$

Number of mini-batches in the epoch: 2 Hint: $\lceil m/b \rceil = \lceil 4/3 \rceil$.

SECTION 12 Comprehensive Comparison

The paths below show the same basic destination but different amounts of update noise. Batch GD moves smoothly, Mini-Batch GD makes small corrections, and SGD makes larger zig-zag movements.



Aspect	Batch GD	Stochastic GD	Mini-Batch GD
Gradient Used	Average of all m	One sample gradient	Average over b samples
Samples/Update	m	1	b
Updates/Epoch	1	m	$\lceil m/b \rceil$
Speed of 1st Update	Slowest (waits for full pass)	Fastest (after 1 sample)	Intermediate (after 1st mini-batch)
Memory Need	High if full batch is materialized	Lowest	Moderate
Stability	Most stable	Least stable	Usually stable enough
Noise	Very low	High (Sampling noise)	Lower noise as b increases
Convergence	Smooth, but may be slow per update	Noisy; may oscillate near optimum	Balanced; common practical choice
Scalability	Weak for massive data	Strong for streaming / huge data	Strong for large datasets
Parallelization	Vectorizable but memory heavy	Poor hardware use per update	Excellent matrix parallelism
GPU Friendliness	May exceed memory	Underuses GPU parallelization	Best fit for GPUs
Typical Use Case	Small datasets, teaching baseline	Online learning, streaming	Deep Learning / large-scale ML

◆ TAKEAWAY

Batch GD is accurate but slow to update; SGD is fast to update but noisy; Mini-Batch GD is the practical compromise that most real systems actually use.

PRACTICE P7 · TRUE OR FALSE

- (a) The loss is guaranteed to decrease after every single SGD update. [F]
Justify: _____
- (b) Mini-Batch GD with a batch size of $b = 1$ is mathematically identical to SGD. [T]
Justify: _____
- (c) If $m = 1000$ and $b = 100$, one epoch of Mini-Batch GD will perform 1000 updates. [F]
Justify: 10 updates per epoch

BY THE END OF THIS SHEET, YOU CAN

- ☐ Explain why Batch GD waits too long to update, and why SGD exists as the natural fix.
- ☐ Perform one epoch of SGD and Mini-Batch GD by hand.
- ☐ Compare BGD, SGD, and Mini-Batch mathematically and visually.
- ☐ Explain why shuffling data is critical for stochastic updates.
- ☐ Choose a suitable method for small data, massive data, deep learning, and streaming data.

Key Terms

Term	Definition
Batch Gradient Descent	Updates parameters using the exact average gradient over the entire dataset.
Stochastic GD (SGD)	Updates parameters immediately using the gradient of a single randomly chosen sample.
Mini-Batch GD	Updates parameters using the average gradient over a small group (b) of samples.
Epoch	One complete pass through all m training examples.
Iteration	One parameter update step.
Shuffling	Randomizing the order of the dataset before each epoch to prevent biased learning patterns.
Sampling Noise	The variance introduced by estimating the full gradient from just one (or a few) samples.
Learning Rate (α)	The step size that scales the gradient before it is subtracted from θ .
Batch Size (b)	The number of samples averaged together for one Mini-Batch GD update.